a smart contract change over the lifetime of an application. You followed the same development steps for both the blind auction and ASK Dapps, strengthening your understanding of the development process using Truffle. By now, you should be familiar with the `truffle compile` and `truffle migrate` commands and with the use of the npm (Node.js-based) web server to complete your end-to-end Dapp development.

In a traditional database, if there are about 10,000 airline transactions, you'll have an integrated database for 10,000-plus rows of data items. In a smart contract, you'll define one row of data items representing the transaction. When a Dapp executes, one transaction at a time is confirmed and recorded on-chain, along with the state changes and events emitted. One of the records may be in block 234567, another in block 234589, and so on. These records collectively form the distributed immutable ledger of the blockchain. As you can see, blockchain is not your traditional database: it is a set of records distributed among the blocks of a blockchain along with valid Txs and data from other unrelated applications deployed on it.

In the ASK Dapp example, you may have noticed that the three functions--`ASKRequest()`, `ASKResponse()`, and `settlePayment()`--are not tied together with `if this then that` code or logic inside the smart contract. But these related functions are tied together by a unique identifier: the `reqID`. The airlines' off-chain applications make decisions about calling these functions. ASK member airlines may develop off-chain applications that use the records on the blockchain for provenance, for evaluation of transaction compliance, or for general data analytics purposes.